

netweevil

netweevil is a Rust-first local network analysis tool for OSM-based routing research.

This repository now includes:

- a shared Rust routing core
- a CLI for reproducible local runs
- a preloadable HTTP API for interactive execution
- an additive GTFS transit processor and pedestrian+transit route command
- an in-repo QGIS plugin for QGIS 3 and QGIS 4 that talks to the API and loads spatial outputs
- explicit manifests and bundles under `.netweevil/`
- local exports to `json`, `csv`, `geojson`, `gpkg`, `parquet`, and `geoparquet`

Via-way turn restriction handling, turn penalties, persisted acceleration bundles, and the preloadable API path are present. See `PROGRESS.md`.

Workspace Layout

- `crates/cli`: **netweevil** CLI entry point
- `crates/api`: preloadable HTTP API
- `crates/transit`: GTFS schedule import and pedestrian+transit routing
- `examples/profiles`: reusable example profiles
- `examples/requests`: route, OD, and matrix inputs
- `qgis_plugin/netweevil_qgis`: QGIS plugin package

Build

```
cargo fmt --all
cargo check
cargo test
```

CLI Quickstart

Import a dataset:

```
cargo run -p netweevil-cli -- dataset import datasets/groningen-260317.osm.pbf --name groningen
```

Validate and compile a profile:

```
cargo run -p netweevil-cli -- profile validate examples/profiles/car_research_v1.yml
cargo run -p netweevil-cli -- profile compile --dataset groningen_2026_03 --profile examples
```

Run a route and write a spatial result QGIS can open directly:

```
cargo run -p netweevil-cli -- analyze route \
  --dataset groningen_2026_03 \
```

```
--profile examples/profiles/car_research_v1.yml \
--request examples/requests/route.json \
--out .netweevil/runs/example-route.geojson
```

Run OD and matrix examples:

```
cargo run -p netweevil-cli -- analyze od \
--dataset groningen_2026_03 \
--profile examples/profiles/car_research_v1.yml \
--pairs examples/requests/od_pairs.csv \
--out .netweevil/runs/example-od.gpkg
```

```
cargo run -p netweevil-cli -- analyze matrix \
--dataset groningen_2026_03 \
--profile examples/profiles/car_research_v1.yml \
--origins examples/requests/matrix_origins.csv \
--destinations examples/requests/matrix_destinations.csv \
--out .netweevil/runs/example-matrix.gpkg
```

Import the OpenOV GTFS feed for a reproducible one-week transit window:

```
curl -L https://gtfs.openov.nl/gtfs-rt/gtfs-openov-nl.zip -o datasets/gtfs-openov-nl.zip
```

```
cargo run -p netweevil-cli -- transit import datasets/gtfs-openov-nl.zip \
--name openov_nl_2026_05_09 \
--service-start 2026-05-09 \
--service-days 7
```

Run a pedestrian+transit route with selectable allowed transit modes:

```
cargo run -p netweevil-cli -- analyze transit-route \
--feed openov_nl_2026_05_09 \
--request examples/requests/transit_openov_groningen.json \
--out .netweevil/runs/example-transit-route.json
```

Inspect persistent state:

```
cargo run -p netweevil-cli -- cache list
```

Current Dataset Format

The current fast path expects freshly imported datasets in the new bundle layout:

- .netweevil/bundles/topology/*.bin
- .netweevil/bundles/names/*.bin
- .netweevil/bundles/acceleration/*.bin
- .netweevil/bundles/metrics/*.bin

Older gzip topology bundles and older dataset imports are no longer part of the supported execution path. Rebuild cached datasets before using the API or

running new analyses:

```
rm -rf .netweevil/datasets .netweevil/bundles/topology .netweevil/bundles/names .netweevil/bundles/acceleration
mkdir -p .netweevil/bundles/topology .netweevil/bundles/names .netweevil/bundles/acceleration

cargo run -p netweevil-cli -- dataset import datasets/groningen-260317.osm.pbf --name groningen_2026_03
cargo run -p netweevil-cli -- profile compile --dataset groningen_2026_03 --profile examples
```

Build A Routing-Only .osm.pbf With Osmium

If you want a smaller source .osm.pbf that only keeps the OSM objects **netweevil** currently needs for routing, you can prefilter it with **osmium**.

The current importer uses:

- routable **way** objects with **highway=***
- ferry **way** objects with **route=ferry** or **ferry=***
- turn-restriction **relation** objects with **type=restriction**
- traffic-signal **node** objects with **highway=traffic_signals**
- referenced way nodes and relation member objects needed to keep the routing topology valid

This means you can safely drop buildings, landuse, addresses, POIs, admin boundaries, and most other non-routing data before import.

This filtered file is meant for **netweevil** routing import, not as a general-purpose OSM extract.

Example script using a polygon extract:

```
#!/usr/bin/env bash
set -euo pipefail

SRC_PBF="${1:?usage: ./make_routing_only_pbf.sh <source.osm.pbf> <region.poly> <output.osm.pbf>}"
POLY="${2:?usage: ./make_routing_only_pbf.sh <source.osm.pbf> <region.poly> <output.osm.pbf>}"
OUT_PBF="${3:?usage: ./make_routing_only_pbf.sh <source.osm.pbf> <region.poly> <output.osm.pbf>}"

TMP_DIR="$(mktemp -d)"
trap 'rm -rf "$TMP_DIR"' EXIT

REGION_PBF="$TMP_DIR/region.osm.pbf"
FILTERS_TXT="$TMP_DIR/netweevil-routing-filters.txt"

cat > "$FILTERS_TXT" <<'EOF'
w/highway
w/route=ferry
w/ferry
r/type=restriction
n/highway=traffic_signals
```

EOF

1. Clip to the study area first.

```
osmium extract \  
  --polygon "$POLY" \  
  "$SRC_PBF" \  
  -o "$REGION_PBF" \  
  -O
```

2. Keep only routing-relevant objects.

By default, osmium tags-filter also keeps referenced objects.

--remove-tags strips tags from referenced non-matching objects to shrink the file further.

while still preserving node coordinates and relation/way references needed by netweevil.

```
osmium tags-filter \  
  --expressions="$FILTERS_TXT" \  
  --remove-tags \  
  "$REGION_PBF" \  
  -o "$OUT_PBF" \  
  -O
```

3. Print a quick summary of the result.

```
osmium fileinfo -e "$OUT_PBF"
```

If you prefer a bounding box instead of a polygon, replace the extract step with:

```
osmium extract \  
  --bbox=min_lon,min_lat,max_lon,max_lat \  
  "$SRC_PBF" \  
  -o "$REGION_PBF" \  
  -O
```

Then import the filtered file normally:

```
cargo run -p netweevil-cli -- dataset import path/to/routing-only.osm.pbf --name groningen_2
```

Notes:

- Keep `r/type=restriction` or you will lose turn restrictions.
- Keep `n/highway=traffic_signals` or traffic-signal turn penalties will stop working.
- Keep ferry ways if your profiles or study area depend on ferry connectivity.
- Do not pass `-R/--omit-referenced` to `osmium tags-filter`; `netweevil` needs the referenced topology objects.

Performant API

This is the recommended interactive path, including QGIS use.

Startup And Network Loading

For the current fast path:

- import the dataset again with the current format
- compile every profile you want available before startup
- start the API with one `--default-profile` and any extra repeatable `--profile` flags

When you run `api serve`, the service loads the dataset manifest, hot topology bundle, persisted acceleration bundle, and one prepared in-memory routing engine per preloaded profile before it starts accepting requests. The edge-name bundle stays cold and is only loaded if you ask for `segment_rows`.

Start the API:

```
cargo run -p netweevil-cli -- api serve \  
  --dataset groningen_2026_03 \  
  --default-profile examples/profiles/car_research_v1.yml \  
  --profile examples/profiles/pedestrian_research_v1.yml \  
  --transit-feed openov_nl_2026_05_09 \  
  --bind 127.0.0.1:8080
```

`api serve` options:

- `--dataset <dataset_id>`: required; selects the imported dataset manifest under `.netweevil/datasets/`
- `--default-profile <path>`: required; default profile loaded at startup and used when requests omit `profile_id`
- `--profile <path>`: optional and repeatable; preload additional selectable profiles at startup
- `--transit-feed <feed_id>`: optional and repeatable; preload imported GTFS transit feeds for `/v1/transit-route`
- `--bind <host:port>`: optional; defaults to `127.0.0.1:8080`

Useful discovery endpoints after startup:

- GET `/healthz`
- GET `/readyz`
- GET `/v1/service`
- GET `/v1/profiles`
- GET `/v1/profiles/{profile_id}`

Response Formats

For POST `/v1/route`, POST `/v1/od`, and POST `/v1/matrix`:

- default response is JSON
- `?format=geojson` switches the response to GeoJSON

Performance tradeoff:

- plain JSON is the warmest path
- `format=geojson` does extra response shaping
- `segment_rows: true` loads cold edge-name data
- `geometry: none` keeps geometry work minimal
- `geometry: full` or `geometry: segments` returns route coordinates and costs more

Pure summary-only route JSON also omits `node_path` and `edge_path`. If you request richer detail, those path arrays are included again.

Request Options

Common route request fields:

- `profile_id`: optional wrapper field on all execution endpoints; uses the service default if omitted
- `request.route_id`: required string
- `request.origin` and `request.destination`: required points with `id`, `lon`, and `lat`
- `request.snap.max_distance_m`: optional; defaults to 500.0
- `request.returns.geometry`: optional; one of `none`, `full`, or `segments`
- `request.returns.segment_rows`: optional boolean
- `request.returns.road_type_breakdown`: optional array of `distance_m` and/or `time_s`
- `request.returns.surface_breakdown`: optional array of `distance_m` and/or `time_s`
- `request.returns.penalty_breakdown`: optional boolean
- `request.returns.explain_cost_derivation`: optional boolean

Current response enrichment is centered on geometry, segment rows, and road/surface breakdowns. The `penalty_breakdown` and `explain_cost_derivation` flags are accepted in the request shape for forward compatibility, but they do not currently add separate response sections.

OD request fields:

- `request.pairs`: array of { `pair_id`, `origin`, `destination` }
- `request.snap.max_distance_m`: optional; defaults to 500.0
- `request.returns.geometry`: optional; use `full` if you want route geometries in successful pair rows

Matrix request fields:

- `request.origins.points`: array of { `id`, `lon`, `lat` }
- `request.destinations.points`: array of { `id`, `lon`, `lat` }
- `request.origins.snap.max_distance_m` and `request.destinations.snap.max_distance_m`: optional; default 500.0
- `request.origins.returns.geometry` and `request.destinations.returns.geometry`: optional; use `full` if you want geometries in successful cells

Rich Geometry Examples

Route with geometry, segments, names, and breakdowns:

```
curl -X POST http://127.0.0.1:8080/v1/route \
-H 'content-type: application/json' \
--data '{
  "profile_id": "car_research_v1",
  "request": {
    "route_id": "rich_route_001",
    "origin": { "id": "a", "lon": 6.5665, "lat": 53.2194 },
    "destination": { "id": "b", "lon": 6.5716, "lat": 53.2148 },
    "snap": { "max_distance_m": 500.0 },
    "returns": {
      "geometry": "full",
      "segment_rows": true,
      "road_type_breakdown": ["distance_m", "time_s"],
      "surface_breakdown": ["distance_m", "time_s"],
      "penalty_breakdown": true,
      "explain_cost_derivation": true
    }
  }
}'
```

The same route as GeoJSON:

```
curl -X POST 'http://127.0.0.1:8080/v1/route?format=geojson' \
-H 'content-type: application/json' \
--data @examples/api/ile_de_france_route_car.json
```

OD with geometries:

```
curl -X POST http://127.0.0.1:8080/v1/od \
-H 'content-type: application/json' \
--data @examples/api/ile_de_france_od.json
```

Matrix with geometries:

```
curl -X POST http://127.0.0.1:8080/v1/matrix \
-H 'content-type: application/json' \
--data @examples/api/ile_de_france_matrix.json
```

Inspect the loaded service metadata:

```
curl http://127.0.0.1:8080/v1/service
curl http://127.0.0.1:8080/v1/profiles
```

QGIS Plugin

The QGIS plugin lives under `qgis_plugin/netweevil_qgis`. Full setup instructions are in `qgis_plugin/README.md`.

Minimal install flow:

1. Start the API:

```
cargo run -p netweevil-cli -- api serve \
  --dataset groningen_2026_03 \
  --default-profile examples/profiles/car_research_v1.yml \
  --profile examples/profiles/pedestrian_research_v1.yml \
  --bind 127.0.0.1:8080
```

2. Symlink the plugin into your QGIS plugin directory.

Linux QGIS example:

```
mkdir -p ~/.local/share/QGIS/QGIS3/profiles/default/python/plugins
ln -s /home/elmeriniemi/stuff/netweevil/qgis_plugin/netweevil_qgis ~/.local/share/QGIS/QGIS3/profiles/default/python/plugins
```

Windows QGIS 4 example for a repo living in WSL:

```
New-Item -ItemType Directory -Force "$env:APPDATA\QGIS\QGIS4\profiles\default\python\plugins\netweevil_qgis"
cmd /c mklink /D "$env:APPDATA\QGIS\QGIS4\profiles\default\python\plugins\netweevil_qgis" "%cd%\netweevil_qgis"
```

3. In QGIS, point the plugin at the API.

- Workspace root: /home/elmeriniemi/stuff/netweevil on Linux, or any Windows-visible folder you want to use for request and response files
- API base URL: http://127.0.0.1:8080
- Response format: JSON or GeoJSON
- Profile: the service default or any profile preloaded by the API

4. Press **Refresh Service**, then use:

- Run Route with output .netweevil/runs/qgis-route.geojson
- Run OD with output .netweevil/runs/qgis-od.gpkg
- Run Matrix with output .netweevil/runs/qgis-matrix.gpkg

The plugin auto-loads spatial outputs into the current QGIS project after successful runs.

Full Windows Example

This section assumes:

- your source repo lives in WSL at /home/elmeriniemi/stuff/netweevil
- you have a Windows-visible mount or symlink at /windownloads
- you want to use Windows QGIS 4

Option A: Use Windows QGIS 4 with the WSL build

This is the simplest path if you are already developing in WSL.

1. Build netweevil in WSL:

```
cd /home/elmeriniemi/stuff/netweevil
cargo build -p netweevil-cli
```

2. Make sure the workspace has the data and outputs you want available to Windows QGIS.

If you want to copy the repo to Windows but skip Rust build output:

```
cd /home/elmeriniemi/stuff/netweevil
mkdir -p /downloads/netweevil
for p in .* *; do
  [ "$p" = "." ] || [ "$p" = ".." ] || [ "$p" = "target" ] || cp -a -- "$p" /downloads/netweevil
done
```

That keeps original data such as `datasets/`, `examples/`, and `qgis_plugin/`.

3. Add the QGIS 4 plugin from Windows PowerShell.

If you want QGIS to use the plugin directly from the WSL repo:

```
New-Item -ItemType Directory -Force "$env:APPDATA\QGIS\QGIS4\profiles\default\python\plugins\netweevil_qgis"
cmd /c mklink /D "$env:APPDATA\QGIS\QGIS4\profiles\default\python\plugins\netweevil_qgis" "%cd%\netweevil_qgis"
```

If you copied the repo to Windows first, point the symlink at the Windows copy instead:

```
New-Item -ItemType Directory -Force "$env:APPDATA\QGIS\QGIS4\profiles\default\python\plugins\netweevil_qgis"
cmd /c mklink /D "$env:APPDATA\QGIS\QGIS4\profiles\default\python\plugins\netweevil_qgis" "%cd%\netweevil_qgis"
```

4. Start QGIS 4 and enable the plugin from Plugins -> Manage and Install Plugins.
5. Start the API in WSL:

```
cd /home/elmeriniemi/stuff/netweevil
cargo run -p netweevil-cli -- api serve \
  --dataset groningen_2026_03 \
  --default-profile examples/profiles/car_research_v1.yml \
  --bind 127.0.0.1:8080
```

6. In the plugin dock, use:

- Workspace root: `\\wsl$\Ubuntu\home\elmeriniemi\stuff\netweevil`
- API base URL: `http://127.0.0.1:8080`
- Profile: `examples/profiles/car_research_v1.yml`

7. Press Refresh Service, then run:

- Route with output `.netweevil/runs/qgis-route.geojson`
- OD with output `.netweevil/runs/qgis-od.gpkg`
- Matrix with output `.netweevil/runs/qgis-matrix.gpkg`

Option B: Use a native Windows build

Use this if you want QGIS to run `netweevil.exe` directly without `ws1.exe`.

1. Copy the repo to Windows.

From WSL:

```
cd /home/elmeriniemi/stuff/netweevil
mkdir -p /windowdownloads/netweevil
for p in .* *; do
  [ "$p" = "." ] || [ "$p" = ".." ] || [ "$p" = "target" ] || cp -a -- "$p" /windowdownloads/netweevil
done
```

2. Open Windows PowerShell, then start the API from the Windows copy:

```
cd C:\path\to\netweevil
cargo run -p netweevil-cli -- api serve --dataset groningen_2026_03 --default-profile examples
```

3. Install the plugin into QGIS 4:

```
New-Item -ItemType Directory -Force "$env:APPDATA\QGIS\QGIS4\profiles\default\python\plugins\netweevil_qgis"
cmd /c mklink /D "$env:APPDATA\QGIS\QGIS4\profiles\default\python\plugins\netweevil_qgis" "C:\path\to\netweevil"
```

4. In QGIS 4, use:

- Workspace root: `C:\path\to\netweevil`
- API base URL: `http://127.0.0.1:8080`
- Profile: `examples/profiles/car_research_v1.yml`

5. Import and run from PowerShell, then use the plugin against that API:

```
cd C:\path\to\netweevil
.\target\debug\netweevil.exe dataset import datasets\groningen-260317.osm.pbf --name groningen_260317
.\target\debug\netweevil.exe profile compile --dataset groningen_2026_03 --profile examples
```

6. In the plugin, press **Refresh Service** and run analyses normally.

Which Windows setup should you use?

- Run the API in WSL if your active repo, datasets, and build artifacts live in WSL.
- Run the API natively on Windows if you want a pure Windows setup.

Input Examples

Route request JSON:

```
{
  "route_id": "route_demo_001",
  "origin": { "id": "origin_a", "lon": 6.5665, "lat": 53.2194 },
  "destination": { "id": "destination_b", "lon": 6.5716, "lat": 53.2148 },
  "snap": { "max_distance_m": 500.0 },
}
```

```
"returns": { "geometry": "full", "segment_rows": true }  
}
```

OD CSV:

```
id,source_x,source_y,target_x,target_y  
od_demo_001,6.5665,53.2194,6.5716,53.2148  
od_demo_002,6.5636,53.2181,6.5698,53.2137
```

Matrix point-set CSV:

```
id,x,y  
origin_a,6.5665,53.2194  
origin_c,6.5636,53.2181
```